Assignment 2(2)

Advanced Scheduler (MLFQS)

20215174 임혁준

20225087 성준호

Submit : 2025.04.14

1. 문제 분석

Problem 1에서 Priority Scheduling을 통해, 기존의 Round-Robin + FIFO 방식보다 실시간 반응이 필요한 작업에 효율적으로 대응할 수 있도록 Scheduling을 개선하였지만, Priority Scheduling 또한 여전히 한계점을 지니고 있다. Priority Scheduling의 Priority는 사용자가 설정하거나 수동으로 바꾸지 않는 이상 CPU를 많이 쓰는 thread든, 거의 안 쓰는 thread든 고정되어 있어 비효율적이고, 자체적으로 priority를 조정하는 기능이 없어 높은 priority의 스레드들이 계속 생성되어, 낮은 priority의 스레드는 영원히 실행되지 못할 수도 있는 Starvation과 같은 문제가 발생할 수 있으며, thread의 CPU 사용량 변화 등에 유동적으로 대응할 수 없어 효율성이 떨어진다. 따라서 이러한 한계를 보완하기 위해 MLFQS (Multi-Level Feedback Queue Scheduler)가 고안되었으며, 이는 thread의 CPU 사용량, 대기 시간, nice value (우선순위 힌트) 등에 따라 우선순위를 동적으로 조절하는 scheduler이다.

2. 알고리즘 설계

스탠포드 대학교의 안내 페이지에 따르면, Pintos는 4.4BSD Scheduler를 고려하여 설계되었다[1]. 따라서 이를 참고하여 Multi-Level Feedback Queue Scheduler를 구성하고자 한다.

MLFQS에서는 기존 Scheduler의 작동과 다르게 priority가 계산되므로, 이를 사용하지 않도록 해야 한다. MLFQS에서 priority는 다음과 같이 계산된다:

$$priority = (PRI\ MAX) - \frac{(recent\ cpu)}{4} - 2 * nice$$

이 계산은 4 tick마다 이루어진다. 추가적으로 priority가 PRI_MIN과 PRI_MAX 범위 내의 값을 가지도록 예외 처리를 해야 한다.

recent_cpu는 각 thread가 cpu를 얼만큼 사용했는 지를 나타내는 값이다. 초기값은 0이며 실행중인 thread의 recent_cpu는 timer interrupt가 발생할 때마다 1씩 증가한다. 일정 tick(PintOS에서는 TIMER_FREQ tick)마다 아래 식에 의해 재계산된다:

$$(recent\ cpu) = \frac{2\ (load\ avg)}{2\ (load\ avg) + 1} * (recent\ cpu) + nice$$

load_avg는 ready 되어있었던 스레드의 평균 개수를 나타내는 값이다:

$$(load\ avg) = \frac{59}{60} * (load\ avg) + \frac{1}{60} ready_threads$$

ready_threads는 idle상태가 아닌 ready상태의 thread의 개수와 실행중인 thread 1개를 합한 값이다. load_avg의 갱신은 TIMER_FREQ tick마다 진행된다.

recent_cpu와 load_avg는 정수가 아닌 실수값을 가지기 때문에 계산은 Fixed-Point를 기반으로 진행되어야 한다. 따라서 계산 과정에서 고정소수점값인 2^14를 적절히 사용해 Fixed Point와 Integer의 변환을 고려해야 한다.

우리는 timer_interrupt 함수에서 위 계산을 적절히 호출해야 한다.

1. 매 틱마다 recent_cpu의 증가
2. TIMER_FREQ틱마다 load_avg와 recent_cpu의 재계산
3. 4틱마다 priority 재계산

추가적으로 우리는 MLFQS 방식일 경우 기존의 thread_set_priority 함수와 donation 기능을 사용하지 않도록 예외처리를 해주어야 한다.

PintOS의 thread_set_nice(), thread_get_nice(), thread_get_load_avg(), thread_get_recent_cpu()는 정의만 되어있고 기능이 미구현되어있어 고정소수점 형식을 유의하여 이들을 구현해야 한다.

최종적으로, 우리가 구현해야 할 내용은 다음과 같다.

1. thread struct에 recent_cpu와 nice 추가, thread.c에 전역변수 load_avg 추가.
2. priority, recent_cpu, load_avg 계산 함수 추가
3. recent_cpu 값 증가 함수 추가
4. 전체 priority와 recent_cpu 갱신 함수 추가.
5. timer_interrpt에 priority, recent_cpu, load_avg 갱신 추가
6. MLFQS 방식일 경우, thread_set_priority가 사용되지 않도록 수정
7. MLFQS 방식일 경우, donation이 사용되지 않도록 수정
8. thread_set_nice(), thread_get_nice(), thread_get_load_avg, thread_get_recent_cpu() 구현

3. 구현

1. thread struct에 recent_cpu와 nice 추가, thread.c에 전역변수 load_avg 추가.

```c
static void
init_thread (struct thread *t, const char *name, int priority)
{
  ASSERT (t != NULL);
  ASSERT (PRI_MIN <= priority && priority <= PRI_MAX);
  ASSERT (name != NULL);

  memset (t, 0, sizeof *t);
  t->status = THREAD_BLOCKED;
  strlcpy (t->name, name, sizeof t->name);
  t->stack = (uint8_t *) t + PGSIZE;
  t->priority = priority;
  t->magic = THREAD_MAGIC;
  list_push_back (&all_list, &t->allelem);

  /* Added for priority donation. */
  t->priority_backup = priority;
  list_init(&t->donations);
  t->waiting_lock = NULL;

  /* Added for mlfqs. */
  t->nice = 0;
  t->recent_cpu = 0;
}
```

thread.c

```c
int load_avg;
void thread_start (void)
{
  load_avg = 0;
  /* Create the idle thread. */
```

thread.c

```c
struct thread
  {
    /* Owned by thread.c. */
    tid_t tid;
    enum thread_status status;
    char name[16];
    uint8_t *stack;
    int priority;
    struct list_elem allelem;


    /* Variable for avoiding b
    int64_t wake_time;

    /* Variables for mlfqs. */
    int nice;
    int recent_cpu;
```

thread.h

2, 3, 4. priority, recent_cpu, load_avg 계산 함수 추가, recent_cpu 값 증가 함수 추가, 전체 priority와 recent_cpu 갱신 함수 추가.

```c
/* Additional functions for mlfqs. */
void mlfqs_priority(struct thread* t);
void mlfqs_recent_cpu(struct thread* t);
void mlfqs_load_avg();
void mlfqs_increment();
void mlfqs_priority_all();
void mlfqs_recent_cpu_all();
```

thread.h

```c
/* Fixed point. */
#define FP (1<<14)
/* Additional functions for mlfqs. */
void
mlfqs_priority(struct thread* t)
{
    if (t == idle_thread) return;
    int result = ((int64_t)PRI_MAX * FP - (int64_t)t->nice * 2 * FP - (t->recent_cpu / 4)) / FP;
    if (result > PRI_MAX) result = PRI_MAX;
    if (result < PRI_MIN) result = PRI_MIN;
    t->priority = result;
}

void
mlfqs_recent_cpu(struct thread* t)
{
    if (t == idle_thread) return;
    int64_t a = ((int64_t)load_avg * 2 * FP / ((int64_t)load_avg * 2 + FP));
    int result = (a * t->recent_cpu) / FP + ((int64_t)t->nice * FP);
    if (result < 0) result = 0;
    t->recent_cpu = result;
}

void
mlfqs_load_avg()
{
    int ready_size = list_size(&ready_list);
    if (thread_current() != idle_thread) ready_size += 1;
    load_avg = (((int64_t)load_avg * 59) / 60) + (((int64_t)ready_size * FP) / 60);
}
void
mlfqs_increment()
{
    struct thread* cur = thread_current();
    if (cur == idle_thread)return;
    cur->recent_cpu = cur->recent_cpu + FP;
}

void
mlfqs_priority_all()
{
    struct list_elem* i;
    for (i = list_begin(&all_list);i != list_end(&all_list);i = list_next(i))
    {
        mlfqs_priority(list_entry(i, struct thread, allelem));
    }
}

void
mlfqs_recent_cpu_all()
{
    struct list_elem* i;
    for (i = list_begin(&all_list);i != list_end(&all_list);i = list_next(i))
    {
        mlfqs_recent_cpu(list_entry(i, struct thread, allelem));
    }
}
```

thread.c

5. timer_interrpt에 priority, recent_cpu, load_avg 갱신 추가

```c
/* Timer interrupt handler. */
static void
timer_interrupt (struct intr_frame *args UNUSED)
{
  ticks++;
  thread_tick ();
  thread_wake(timer_ticks ());

  if (!thread_mlfqs) return;
  mlfqs_increment();
  if (ticks % 4 == 0)
  {
      mlfqs_priority_all();
      if (ticks % TIMER_FREQ == 0)
      {
          mlfqs_load_avg();
          mlfqs_recent_cpu_all();
      }
  }
}
```

timer.c

6. MLFQS 방식일 경우, thread_set_priority가 사용되지 않도록 수정

```c
/* Sets the current thread's priority to NEW_PRIORITY. */
void
thread_set_priority (int new_priority)
{
    if (thread_mlfqs) return;
    struct thread* cur = thread_current();

    cur->priority_backup = new_priority;
    cur->priority = new_priority;
    if (!list_empty(&cur->donations))
    {
        struct thread* thread_top = list_entry(list_front(&cur->donations), struct thread, donation_elem);
        if (cur->priority < thread_top->priority)
            cur->priority = thread_top->priority;
    }

    thread_check_preemption();
}
```

thread.c

7. MLFQS 방식일 경우, donation이 사용되지 않도록 수정

```c
void
lock_acquire (struct lock *lock)
{
  ASSERT (lock != NULL);
  ASSERT (!intr_context ());
  ASSERT (!lock_held_by_current_thread (lock));

  struct thread* cur = thread_current();
  if (lock->holder != NULL)
  {
      cur->waiting_lock = lock;
      list_insert_ordered(&lock->holder->donations, &cur->donation_elem, compare_donation, NULL);
      if (!thread_mlfqs) priority_donation();
  }
  sema_down(&lock->semaphore);
  cur->waiting_lock = NULL;
  lock->holder = cur;
}
void
lock_release (struct lock *lock)
{
    ASSERT(lock != NULL);
    ASSERT(lock_held_by_current_thread(lock));

    struct thread* cur = thread_current();

    if (!thread_mlfqs)
    {
        struct list_elem* i;
        for (i = list_begin(&cur->donations); i != list_end(&cur->donations); i = list_next(i))
        {
            struct thread* thread_i = list_entry(i, struct thread, donation_elem);
            if (thread_i->waiting_lock == lock)
                list_remove(&thread_i->donation_elem);
        }

        cur->priority = cur->priority_backup;
        if (!list_empty(&cur->donations))
        {
            list_sort(&cur->donations, compare_donation, NULL);
            struct thread* thread_top = list_entry(list_front(&cur->donations), struct thread, donation_elem);
            if (cur->priority < thread_top->priority)
                cur->priority = thread_top->priority;
        }
    }

    lock->holder = NULL;
    sema_up(&lock->semaphore);
}
```

synch.c

8. thread_set_nice(), thread_get_nice(), thread_get_load_avg, thread_get_recent_cpu() 구현

```c
/* Sets the current thread's nice value to NICE. */
void
thread_set_nice (int nice)
{
    struct thread* cur = thread_current();
    enum intr_level old_level = intr_disable();
    cur->nice = nice;
    mlfqs_priority(cur);

    thread_check_preemption();

    intr_set_level(old_level);
}

/* Returns the current thread's nice value. */
int
thread_get_nice (void)
{
    enum intr_level old_level = intr_disable();
    int result = thread_current()->nice;
    intr_set_level(old_level);
    return result;
}

/* Returns 100 times the system load average. */
int
thread_get_load_avg (void)
{
    enum intr_level old_level = intr_disable();
    int result = (load_avg * 100) / FP;
    intr_set_level(old_level);
    return result;
}

/* Returns 100 times the current thread's recent_cpu value. */
int
thread_get_recent_cpu (void)
{
    enum intr_level old_level = intr_disable();
    int result = (thread_current()->recent_cpu * 100) / FP;
    intr_set_level(old_level);
    return result;
}
```

thread.c

4. 결과

- make test 실행결과

```
pass tests/threads/alarm-single
pass tests/threads/alarm-multiple
pass tests/threads/alarm-simultaneous
pass tests/threads/alarm-priority
pass tests/threads/alarm-zero
pass tests/threads/alarm-negative
pass tests/threads/priority-change
pass tests/threads/priority-donate-one
pass tests/threads/priority-donate-multiple
pass tests/threads/priority-donate-multiple2
pass tests/threads/priority-donate-nest
pass tests/threads/priority-donate-sema
pass tests/threads/priority-donate-lower
pass tests/threads/priority-fifo
pass tests/threads/priority-preempt
pass tests/threads/priority-sema
pass tests/threads/priority-condvar
pass tests/threads/priority-donate-chain
pass tests/threads/mlfqs-load-1
pass tests/threads/mlfqs-load-60
pass tests/threads/mlfqs-load-avg
pass tests/threads/mlfqs-recent-1
pass tests/threads/mlfqs-fair-2
pass tests/threads/mlfqs-fair-20
pass tests/threads/mlfqs-nice-2
pass tests/threads/mlfqs-nice-10
pass tests/threads/mlfqs-block
All 27 tests passed.
```

5. 참고 문헌

[1] *Pintos Projects: 4.4BSD Scheduler*. (2025). Stanford.edu. https://web.stanford.edu/class/cs140/projects/pintos/pintos_7.html